

Clause-Activation Signatures for Attack-Family Attribution in IoMT Using Tsetlin Machines

Rakesh Reddy Yakkati, Per-Arne Andersen, Rishad Shafik, Ole-Christoffer Granmo and Lei Jiao

Abstract—Attack-family attribution in the Internet of Medical Things (IoMT) requires accurate predictions and evidence that analysts can inspect. We introduce Clause-Activation Signatures (CAS) to extract signed clause evidence from a trained, weighted Tsetlin Machine (TM). Weight-based pruning reduces the candidate pool before complementary-pairs stability selection identifies repeatedly selected clauses. Averaged coefficient signs distinguish supporting from opposing evidence. We evaluate CAS on capped CICIoMT2024 WiFi/MQTT data. Separate training partitions support TM fitting, signature extraction, and operating-point selection. The test set is reserved for final evaluation. Across three random seeds, the full 2,400-clause TM achieves macro-F1 0.6660 ± 0.0125 . After pruning, CAS achieves 0.6660 ± 0.0316 at $\pi_{thr} = 0.8$ and 0.6599 ± 0.0371 at $\pi_{thr} = 0.9$. Including both evidence signs, these settings use 216 and 193 distinct clauses on average, respectively. This is an 11.1 to 12.4-fold reduction from the full pool, with similar mean macro-F1. Replacing clause activations with the 154 input literals under the same readout lowers macro-F1 to 0.5876 ± 0.0112 . Five standard classifiers using the same underlying features achieve 0.6284 to 0.6733. Both highlighted CAS settings fall within this range. CAS provides compact clause evidence with explicit signs. We also derive a sufficient condition for pruning to preserve TM predictions and clarify the scope of stability-selection error control. Evaluation on one capped dataset leaves generalisation to other settings open.

Index Terms—attack attribution, intrusion detection, interpretable machine learning, IoMT, stability selection, Tsetlin Machine

I. INTRODUCTION AND RELATED WORK

Attack-family attribution identifies malicious traffic types in the Internet of Medical Things (IoMT). These labels help analysts interpret security alerts. Neural classifiers can perform this task. For example, Yin et al. [1] used recurrent neural networks to distinguish normal traffic and four attack categories on NSL-KDD. To explain predictions, SHAP [2] and LIME [3] attribute contributions to input features [4]. Analysts can also inspect the logical rules learned by the detector itself.

The Tsetlin Machine (TM) learns conjunctive rules over Boolean literals [5]. Weighted variants associate learned weights with clauses to obtain more compact model representations [6]. Abeyrathna et al. [7] applied TM rules to intrusion detection on KDD'99. Their model provided both predictions and inspectable rules. More recently, Jaiswal et al. [8] evaluated a TM detector on CICIoMT2024 and explained

individual predictions through class-wise vote scores and clause-activation heatmaps. Their on-device IoMT study [9] used MedSec-25, examined feature contributions and clause activations, and evaluated inference on a Raspberry Pi. These studies establish both rule-based detection and inspection of the clauses contributing to a prediction.

Building on this work, we introduce Clause-Activation Signatures (CAS) to select compact, reusable class signatures from a trained clause pool. A clause's TM vote alone does not show its additional value for distinguishing attack families. It also does not show how consistently the clause is selected across subsamples. Weight-based pruning first reduces the candidate pool. CAS then applies complementary-pairs stability selection (CPSS) [10], [11] to the retained clauses' activations. The frozen TM produces these activations on held-out training flows. Averaged coefficients from one-versus-rest models identify supporting and opposing evidence for each class. These evidence signs are estimated separately from the original TM voting signs. CAS scores each flow by normalizing the signed evidence from selected clauses that activate.

We evaluate whether CAS reduces the evidence set while retaining predictive performance. Our evaluation uses per-class sampling caps without SMOTE and separate partitions for TM fitting, signature extraction, and operating-point selection. This sampling and partitioning protocol differs from prior CICIoMT2024 work [8]. The caps also change the test class distribution. This can affect macro-F1 through per-class precision, despite equal class weighting. These differences prevent a direct performance ranking against their published results. We therefore compare CAS with the full TM and standard classifiers evaluated within this study on a common report set.

Our contributions are:

- A clause-based attribution layer that combines CPSS with signed, normalized evidence to construct reusable class signatures.
- A sufficient margin condition for prediction-preserving TM pruning and an explicit account of the scope of CPSS error control.

II. METHOD: CLAUSE-ACTIVATION SIGNATURES

A. Booleanisation and Weighted TM

Figure 1 summarizes the pipeline. Training-fitted quantile bins encode numeric features as cumulative thermometer bits; non-finite values are replaced with the finite training maximum. TMU [12] supplies literal negation internally. For N_{cls}

Supported by the Research Council of Norway (SecureIoTM, grant 342167, TEKNOLOGIKONVER), with support from CAIR at the University of Agder, Sørlandet Hospital Trust, and Newcastle University.

R. R. Yakkati, L. Jiao, and O.-C. Granmo: Dept. of ICT, University of Agder, Grimstad, Norway; P.-A. Andersen: Dept. of Information Systems, University of Agder, Kristiansand, Norway; R. Shafik: Dept. of Microelectronic Systems, Newcastle University, UK; Contact: rakeshry@uia.no.

classes and C clauses per class, the trained pool produces Boolean activations $z_{c,j}(x)$ and signed class sums

$$f_c(x) = \sum_{j=1}^C w_{c,j} z_{c,j}(x), \quad \hat{y}_{\text{TM}}(x) = \arg \max_c f_c(x). \quad (1)$$

Flattening class and clause indices gives the activation matrix $Z \in \{0, 1\}^{n \times p}$, initially with $p = N_{\text{cls}} C$.

B. Clause-Pool Pruning

Let $m_{c,j} \in \{0, 1\}$ retain a clause and $f_c(x; m) = \sum_j m_{c,j} w_{c,j} z_{c,j}(x)$. The target is a small pool whose macro-F1 on the selection partition remains within ε of the full model:

$$\min_m \|m\|_0 \quad \text{s.t.} \quad F_{1,\text{sel}}(m) \geq F_{1,\text{sel}}(\mathbf{1}) - \varepsilon. \quad (2)$$

We use a magnitude surrogate rather than solve this discrete objective. For removed indices D_c , set $R_c = \sum_{j \in D_c} |w_{c,j}|$ and $R = \max_c R_c$. Since activations are Boolean,

$$|f_c(x) - f_c(x; m)| \leq R_c \leq R. \quad (3)$$

If $a = \hat{y}_{\text{TM}}(x)$ and $\gamma(x) = f_a(x) - \max_{c \neq a} f_c(x)$, the margin of a over any competing class decreases by at most $2R$. Thus

$$\gamma(x) > 2R \implies \arg \max_c f_c(x; m) = \hat{y}_{\text{TM}}(x). \quad (4)$$

For a fixed deletion budget k per class, the smallest-magnitude weights minimize this surrogate:

$$D_c^*(k) = \arg \min_{|D|=k} \sum_{j \in D} |w_{c,j}|. \quad (5)$$

C. CPSS and Signed Evidence

For each class ℓ , CPSS divides the fit partition into B complementary pairs as shown in Fig 1. On each of the $2B$ halves, a class-balanced ℓ_1 logistic model predicts the one-versus-rest class label from Z . For half h and inverse-regularisation strength $\kappa \in \Lambda$, let $\beta_{\ell j}^{h,\kappa}$ be the fitted coefficient. A clause receives one vote if any grid point selects it, using numerical tolerance δ :

$$\hat{\pi}_{\ell j} = \frac{1}{2B} \sum_{h=1}^{2B} \mathbf{1} \left\{ \max_{\kappa \in \Lambda} |\beta_{\ell j}^{h,\kappa}| > \delta \right\}. \quad (6)$$

Let $\bar{\beta}_{\ell j}$ average coefficients over all halves and grid points, including zeros. The signed supports are

$$S_{\ell}^{\pm} = \{j : \hat{\pi}_{\ell j} \geq \pi_{\text{thr}}, \quad \pm \bar{\beta}_{\ell j} > 0\}. \quad (7)$$

Positive coefficients provide inculpatory evidence; negative ones provide exculpatory evidence. The attribution rule is

$$s_{\ell}(x) = \frac{\sum_{j \in S_{\ell}^+} \hat{\pi}_{\ell j} z_j(x) - \sum_{j \in S_{\ell}^-} \hat{\pi}_{\ell j} z_j(x)}{\sum_{j \in S_{\ell}^+} \hat{\pi}_{\ell j}}. \quad (8)$$

Prediction uses $\hat{y}(x) = \arg \max_{\ell} s_{\ell}(x)$.

D. Scope of Stability Error Control

For a fixed pool and selector, let p_j denote a clause's population selection probability on a half-sample, $L_{\theta} = \{j : p_j \leq \theta\}$, and q the expected number selected per half. Under the sampling conditions of CPSS [11], the sign-agnostic support S_{ℓ} satisfies, for $\pi_{\text{thr}} > 1/2$,

$$\mathbb{E}|S_{\ell} \cap L_{\theta}| \leq \frac{\theta q}{2\pi_{\text{thr}} - 1}. \quad (9)$$

Taking $\theta = q/p$ yields $q^2/((2\pi_{\text{thr}} - 1)p)$. This controls low-selection-probability inclusions. Interpreting them as false discoveries requires additional assumptions; using $\hat{q} = \sum_j \hat{\pi}_{\ell j}$ is a plug-in diagnostic, not a validated false-discovery certificate. Correlated flows, adaptive operating-point selection, and coefficient signs require separate consideration. No finite bound is claimed at $\pi_{\text{thr}} = 0.5$.

III. EXPERIMENTAL SETUP AND RESULTS

CICIoMT2024 contains 18 attacks on 40 devices, including 25 real and 15 simulated devices [13]. We use its WiFi/MQTT flow features with six labels: Benign, DDoS, DoS, MQTT, Recon, and Spoofing. The selected features exclude IP addresses and port fields. Sampling is capped at 40,000 training and 15,000 test rows per class. The training set contains 195,338 rows (19,291 Benign, 16,047 Spoofing, 40,000 in each remaining class); the test set contains 80,868 rows (5,868 Spoofing and 15,000 in each other class).

A stratified split of the training set creates 136,735 TM-fit rows, 39,068 CPSS-fit rows, and 19,535 selection rows. The test set serves as the report set for final evaluation. The TM remains frozen during signature extraction, and the CPSS-fit flows are withheld from TM training.

The 22 numeric features use up to ten quantile bins, giving a cached 154-bit representation that includes redundant bits from repeated quantiles. The TM uses $C = 400$ clauses per class (200 positive and 200 negative), $T = 320$, $s = 8$, and 25 epochs. CPSS uses class-balanced liblinear fits with 200 maximum iterations, $\delta = 10^{-8}$, and $\Lambda = \text{geomspace}(0.001, 0.1, 6)$. Pruning sweeps $k \in \{0, 5, \dots, 395\}$ with $\varepsilon = 0.01$. For each pool, the operating point (B, π_{thr}) is chosen to maximize mean selection macro-F1 over $B \in \{5, 10, 15, 20, 25\}$ and $\pi_{\text{thr}} \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$.

A. Closed-Set Attribution and Sign Validation

Table I compares predictive performance and evidence size. For CAS, size is the mean positive-evidence support summed across classes. It excludes opposing evidence, and a clause selected for multiple classes is counted in each class's support.

For the pruned pool, selection-set macro-F1 chooses $(B, \pi_{\text{thr}}) = (10, 0.5)$. This setting achieves report-set macro-F1 0.6755 with 190 positive-evidence clauses. We also highlight (5, 0.8) and (10, 0.9) to illustrate the trade-off at higher selection thresholds and smaller positive supports. At (5, 0.8), CAS uses 149 positive-evidence clauses and matches the full TM's mean macro-F1 to the reported precision. At $\pi_{\text{thr}} = 0.9$, this count falls to 126 and mean macro-F1 decreases by

SIGNATURE EXTRACTION

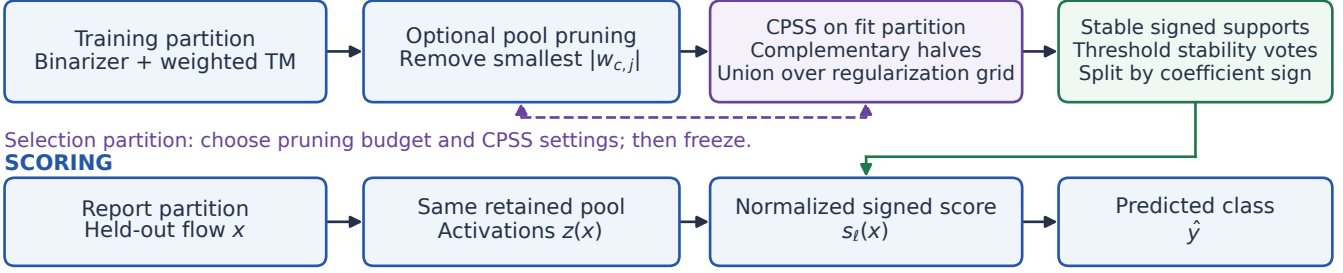


Fig. 1. CAS pipeline with disjoint TM-fit, CPSS-fit, selection, and report partitions.

TABLE I

REPORT-SET PERFORMANCE (THREE SEEDS). CLAUSE COUNTS DENOTE TM POOL SIZE OR CAS POSITIVE SUPPORTS SUMMED ACROSS CLASSES.

Method	(B, π_{thr})	Clauses	macro-F1	Accuracy
TM, full pool	—	2,400	0.6660 ± 0.0125	0.7033
TM, pruned pool	—	360	0.6615 ± 0.0115	0.6982
CAS, full pool	(10, 0.9)	154	0.6408 ± 0.0168	0.6676
CAS, pruned pool	(10, 0.9)	126	0.6599 ± 0.0371	0.6972
CAS, pruned pool	(5, 0.8)	149	0.6660 ± 0.0316	0.7037
CAS, pruned pool	(10, 0.5)	190	0.6755 ± 0.0183	0.7126
Input literals, same readout	(10, 0.9)	—	0.5876 ± 0.0112	0.6271

0.0061, less than the reported standard deviations across seeds. Including opposing evidence, these two settings use 216 and 193 distinct clauses on average: 11.1–12.4 times fewer than the full pool. Their summed positive/negative supports are 149/199 and 126/173, respectively. Clauses shared across classes explain why the distinct counts are smaller than these sums. The selected (10, 0.5) setting uses 251 distinct signed clauses. All counts are rounded means over three seeds. Figure 2 plots the trade-off using positive-support counts.

Two ablations examine what makes the selected evidence useful. Replacing clause activations with the 154 input literals under the same readout yields macro-F1 0.5876 ± 0.0112 . This result supports the value of the clauses’ conjunctive structure. A sign-agnostic scorer normalizes the stability-weighted sum over all selected clauses. It yields macro-F1 0.025 ± 0.032 . This drop shows that the readout depends on distinguishing supporting from opposing evidence.

B. Comparison with Standard Classifiers

Table II compares CAS with five standard classifiers. Each baseline receives the 22 numeric features in their original continuous form. The TM uses thermometer encoding. We fit all baselines on the union of the TM-fit and CPSS-fit partitions. This gives them the 175,803 labelled rows CAS uses across its two stages. We choose hyperparameters on the selection partition and evaluate each model once on the report set. The 300-tree random forest [14] has the highest mean macro-F1 among these baselines, 0.6733. XGBoost follows at 0.6640, the decision tree at 0.6526, the binarized neural network at 0.6503, and the ℓ_1 linear model at 0.6284. Standard deviations

TABLE II

STANDARD CLASSIFIERS ON THE 22 NUMERIC FEATURES, THREE-SEED MEANS ON THE REPORT SET.

Model	macro-F1	Accuracy
Random forest [14]	0.6733	0.7251
XGBoost [16]	0.6640	0.7112
Decision tree	0.6526	0.6972
Binarized neural network [15]	0.6503	0.6697
ℓ_1 logistic regression [17]	0.6284	0.6465

across seeds are below 0.002 for the tree models, 0.011 for the linear model, and 0.043 for the BNN. XGBoost gives identical results across seeds in this experiment. It uses no row or column subsampling.

These baselines provide several points of comparison. A single decision tree and an ℓ_1 linear model serve as interpretable references. Bagged and boosted tree ensembles provide black-box references. The binarized neural network [15] provides a neural comparison that uses bitwise operations, as does the TM. The BNN reaches 0.6503 using 106,496 binary weights. CAS attains 0.6660 from 216 distinct signed clauses. These counts describe different model components; they do not establish relative memory or runtime costs.

C. Sensitivity Before and After Pruning

To assess sensitivity to the CPSS settings, we vary the selection threshold and pair count. At $B = 15$, sweeping π_{thr} over $\{0.5, \dots, 0.9\}$ spans macro-F1 0.656–0.674 on the pruned pool and 0.633–0.652 on the full pool. At $\pi_{\text{thr}} = 0.8$, varying B from 5 to 25 gives ranges of 0.656–0.666 and 0.628–0.641, respectively. Figure 3 compares these sweeps with the pruned-TM baselines. In these sweeps, CAS performs better when selection starts from the pruned pool. This suggests that removing low-weight clauses helps signature extraction in this setting. Across the pruned-pool operating points, positive-evidence support sizes range from 122 to 196 (Fig. 2).

D. Pruning and Signature Size

To examine the effect of pruning alone, Fig. 4 shows TM performance across removal budgets. Report-set macro-F1 and accuracy remain approximately steady as the pool shrinks from 2,400 to roughly 360 clauses, then decline with further

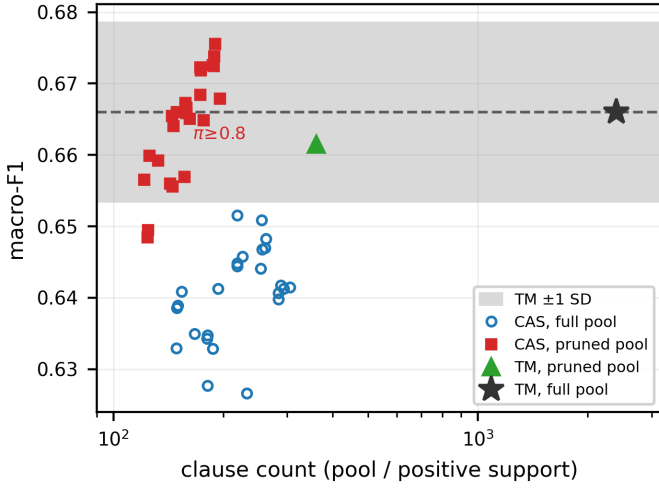


Fig. 2. CAS size-performance trade-off. The band is TM mean $\pm$ SD; 23 of 25 pruned-pool settings lie within it.

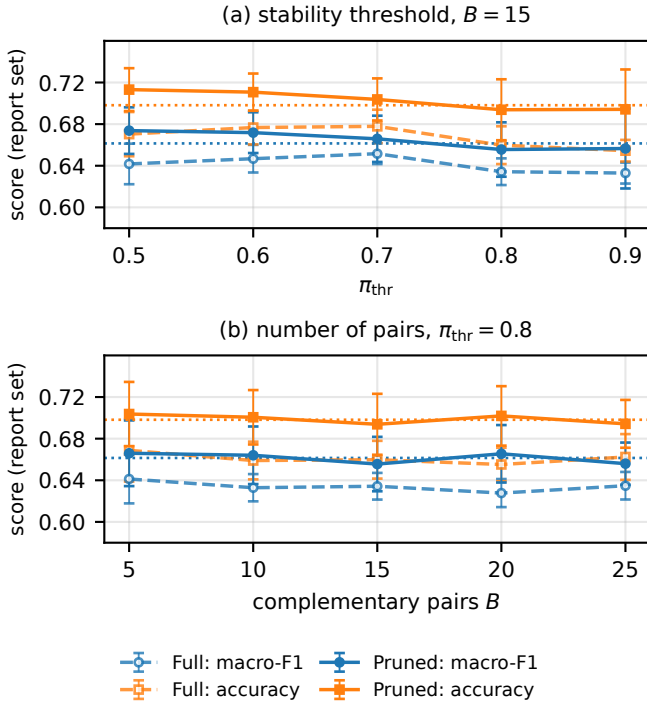


Fig. 3. CAS sensitivity (three-seed mean $\pm$ SD): (a) threshold at $B = 15$; (b) pair count at $\pi_{thr} = 0.8$. Dotted lines: pruned TM.

removal. Using only the selection partition to choose the pruning budget gives $k^* = 340$. This leaves 60 clauses per class and 360 in total, a reduction of 85%. At that budget the pruned TM achieves macro-F1 0.6615 ± 0.0115 against 0.6660 ± 0.0125 for the full pool (Table I). Pruning therefore removes most of the pool with a mean macro-F1 decrease of 0.0045.

Table III shows both evidence signs at each pool's selected setting for seed 42. Summed positive/negative supports are

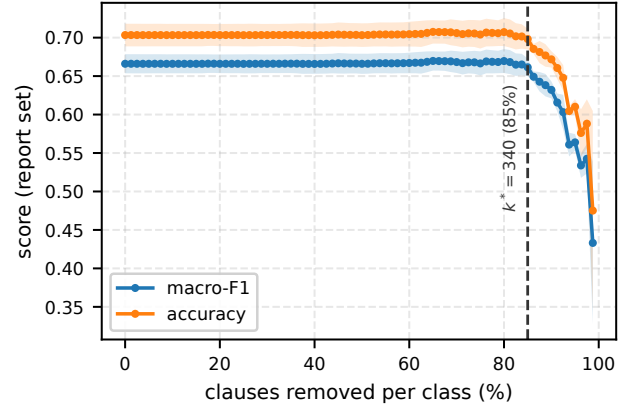


Fig. 4. Pruning sensitivity (mean $\pm$ SD); the dashed line marks $k^* = 340$, chosen on the selection set.

150/585 for the full pool and 185/233 after pruning. These CAS signs are estimated from the pooled clauses across all six classes and differ from the original TM polarities. Shared clauses are counted in each class. The selected thresholds differ, so pruning need not reduce the summed supports.

TABLE III
CAS SUPPORTS, SEED 42: FULL (10, 0.9); PRUNED (10, 0.5).

Class	Full pool		Pruned pool	
	$ S^+ $	$ S^- $	$ S^+ $	$ S^- $
Benign	21	65	35	37
DDoS	29	130	29	39
DoS	21	180	38	40
MQTT	21	67	17	20
Recon	19	47	27	54
Spoofing	39	96	39	43
Total	150	585	185	233

IV. CONCLUSION

CAS extracts reusable, signed clause evidence for IoMT attack-family attribution. On capped CICIoMT2024 data, pruning removes 85% of the TM pool. CAS matches the full TM's mean macro-F1 of 0.6660 with 216 distinct signed clauses. Using 193 yields 0.6599. Ablations support the use of conjunctive rules and evidence signs. Both highlighted settings fall within the baseline performance range. Their compact evidence can be traced to the selected clauses.

The pruning bound depends on the stated margin condition. Under its sampling assumptions, CPSS controls low-selection-probability inclusions, without certifying false discoveries or signs. Further datasets, analyst studies, and runtime measurements are needed to assess generalisation, explanation value, and deployment savings.

REFERENCES

- [1] C. Yin, Y. Zhu, J. Fei, and X. He, “A deep learning approach for intrusion detection using recurrent neural networks,” *IEEE Access*, vol. 5, pp. 21 954–21 961, 2017.
- [2] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.
- [3] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should I trust you?”: Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1135–1144.
- [4] P. Hermosilla, S. Berríos, and H. Allende-Cid, “Explainable AI for forensic analysis: A comparative study of SHAP and LIME in intrusion detection models,” *Applied Sciences*, vol. 15, no. 13, p. 7329, 2025.
- [5] O.-C. Granmo, “The tsetlin machine – a game theoretic bandit driven approach to optimal pattern recognition with propositional logic,” 2018.
- [6] A. Phoulady, O.-C. Granmo, S. Rahimi Gorji, and H. Ahmady Phoulady, “The weighted tsetlin machine: Compressed representations with weighted clauses,” 2020.
- [7] K. D. Abeyrathna, H. S. G. Pussewelage, S. N. Ranasinghe, V. A. Oleschuk, and O.-C. Granmo, “Intrusion detection with interpretable rules generated using the tsetlin machine,” in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2020, pp. 1121–1130.
- [8] R. Jaiswal, P.-A. Andersen, L. R. Cenkeramaddi, L. Jiao, and O.-C. Granmo, “A tsetlin machine-driven intrusion detection system for next-generation IoMT security,” in *2026 Silicon Valley Cybersecurity Conference (SVCC)*. IEEE, 2026, pp. 1–8.
- [9] —, “On-device interpretable tsetlin machine-based intrusion detection for secure IoMT,” 2026, accepted at the IEEE International Conference on Intelligence and Security Informatics (ISI 2026).
- [10] N. Meinshausen and P. Bühlmann, “Stability selection,” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 72, no. 4, pp. 417–473, 2010.
- [11] R. D. Shah and R. J. Samworth, “Variable selection with error control: Another look at stability selection,” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 75, no. 1, pp. 55–80, 2013.
- [12] Centre for Artificial Intelligence Research (CAIR), “Tsetlin machine unified (TMU),” <https://github.com/cair/tmu>.
- [13] S. Dadkhah, E. C. Pinto Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, and A. A. Ghorbani, “CICIoMT2024: A benchmark dataset for multi-protocol security assessment in IoMT,” *Internet of Things*, vol. 28, p. 101351, Dec. 2024.
- [14] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [15] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized neural networks,” in *Advances in Neural Information Processing Systems 29 (NIPS)*, 2016, pp. 4107–4115.
- [16] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.
- [17] R. Tibshirani, “Regression shrinkage and selection via the lasso,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.